

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Estructuras de Datos
Ing. Edgar Ornelis
Ing. Álvaro Hernández
Ing. Luis Espino
Aux. Steven Mejía
Aux. Luis García
Aux. Marcos Itzep



Manual de Usuario - EDDMail

Sistema de Simulación de Correos Electrónicos

Fase 3 - Blockchain, Merkle Tree y Compresión

José Alexander López López

Carné: 202100305

Sección: C

Fecha: 24 de octubre de 2025

Índice

1. Introducción	5
1.1. Novedades de la Fase 3	5
1.2. Características Principales	5
2. Requisitos del Sistema	5
2.1. Requisitos de Hardware	5
2.2. Requisitos de Software	6
2.3. Instalación de Dependencias	6
3. Instalación y Configuración	6
3.1. Obtener el Código Fuente	6
3.2. Compilación	6
4. Capturas de Pantalla del Sistema	6
4.1. Pantalla de Inicio de Sesión	8
4.2. Creación de Cuenta	8
4.3. Panel de Administración (Root)	8
4.4. Bandeja de Entrada	8
4.5. Borradores (Árbol AVL)	8
4.6. Favoritos (Árbol B)	8
4.7. Comunidades (Árbol BST)	8
4.8. Otras Funcionalidades	8
4.9. Reportes del Sistema	8
4.10. Nuevas Capturas - Fase 3	8
4.10.1. Visualización de Blockchain	8
4.10.2. Verificación de Árbol de Merkle	8
4.10.3. Compresión LZW en Correos	8
4.10.4. Control de Logueo	8
4.10.5. Grafo de Contactos (Integración Grupal)	8
5. Interfaz de Usuario	8
5.1. Pantalla de Inicio de Sesión	8
5.2. Creación de Nueva Cuenta	9
6. Usuario Root (Administrador)	10
6.1. Panel de Administración	10
6.1.1. Carga Masiva de Usuarios	10
6.1.2. Carga Masiva de Correos	11
6.1.3. Carga Masiva de Contactos (NUEVO - Fase 3)	11
6.1.4. Gestión de Comunidades (Árbol BST)	12
6.2. Reportes del Administrador	13
7. Usuario Estándar	13
7.1. Bandeja de Entrada	13
7.2. Borradores (Árbol AVL)	14
7.2.1. Guardar Borrador	15
7.2.2. Visualizar Borradores	15

7.2.3.	Modificar y Enviar	16
7.3.	Favoritos (Árbol B de Orden 5)	16
7.3.1.	Marcar como Favorito	16
7.3.2.	Gestionar Favoritos	17
7.4.	Comunidades (Árbol BST)	17
7.4.1.	Unirse a Comunidad	18
7.4.2.	Publicar en Comunidad	18
7.4.3.	Ver Mensajes	18
7.4.4.	Reaccionar a Mensajes (NUEVO - Fase 3)	19
7.5.	Otras Funcionalidades	20
7.5.1.	Enviar Correo	20
7.5.2.	Gestión de Contactos	20
7.5.3.	Papelera	21
7.5.4.	Correos Programados	21
7.5.5.	Actualizar Perfil	22
8.	Nuevas Funcionalidades - Fase 3	22
8.1.	Blockchain	22
8.1.1.	Arquitectura del Blockchain	22
8.1.2.	Bloque Génesis	23
8.1.3.	Minería de Bloques	24
8.1.4.	Visualización del Blockchain	25
8.1.5.	Reporte Graphviz	26
8.2.	Árbol de Merkle	27
8.2.1.	Construcción del Árbol	27
8.2.2.	Propiedades del Árbol de Merkle	28
8.2.3.	Nodos del Árbol	29
8.2.4.	Verificación de Integridad	30
8.2.5.	Búsqueda en Árbol de Merkle	31
8.2.6.	Reporte Graphviz	32
8.3.	Compresión LZW	33
8.3.1.	Algoritmo LZW	33
8.3.2.	Descargar Correo Comprimido	34
8.3.3.	Formato del Archivo Comprimido	35
8.3.4.	Tasas de Compresión	36
8.3.5.	Casos de Uso	36
8.4.	Control de Logueo	36
8.4.1.	Estructura del Registro	37
8.4.2.	Registro Automático	37
8.4.3.	Visualización del Control de Logueo	37
8.4.4.	Búsqueda en Logueo	38
8.4.5.	Exportación JSON	38
8.4.6.	Reporte Graphviz	39
8.4.7.	Casos de Uso	39
8.5.	Grafo de Contactos (Integración Grupal)	39
8.5.1.	Estructura del Grafo	39
8.5.2.	Construcción del Grafo	40
8.5.3.	Carga Masiva de Contactos	40

8.5.4.	Visualización del Grafo	41
8.5.5.	Análisis del Grafo	41
8.5.6.	Generación del Reporte	42
9.	Estructuras de Datos	42
9.1.	Fase 1 (Mantenidas)	42
9.2.	Fase 2 (Mantenidas)	42
9.3.	Fase 3 (Nuevas)	42
9.3.1.	Blockchain	42
9.3.2.	Árbol de Merkle	43
9.3.3.	Grafo No Dirigido	43
9.3.4.	Lista Enlazada Simple (Logueo)	43
10.	Tabla de Complejidades	44
11.	Sistema de Reportes	44
11.1.	Reportes Root	44
11.2.	Reportes Usuario	44
12.	Ejemplos de Uso - Fase 3	45
12.1.	Ejemplo 1: Enviar Correo con Blockchain	45
12.2.	Ejemplo 2: Verificar Integridad con Merkle	45
12.3.	Ejemplo 3: Descargar Correo Comprimido	46
12.4.	Ejemplo 4: Auditoría con Control de Logueo	46
12.5.	Ejemplo 5: Carga Masiva de Contactos (Grupal)	47
12.6.	Ejemplo 6: Reaccionar a Mensaje en Comunidad	47
13.	Preguntas Frecuentes - Fase 3	48
13.1.	¿Qué es el Blockchain y por qué se usa?	48
13.2.	¿Cómo funciona el proof-of-work?	48
13.3.	¿Qué ventaja tiene el Árbol de Merkle sobre Árbol B?	48
13.4.	¿Cuánto comprime realmente LZW?	49
13.5.	¿El Control de Logueo afecta rendimiento?	49
13.6.	¿Por qué Grafo no dirigido para contactos?	49
13.7.	¿Se puede modificar un bloque en el blockchain?	49
13.8.	¿Merkle Tree se reconstruye cada vez?	50
14.	Solución de Problemas - Fase 3	50
14.1.	Blockchain muy lento	50
14.2.	Merkle no detecta cambios	50
14.3.	Compresión LZW falla	50
14.4.	Control de Logueo no registra salidas	51
14.5.	Grafo de contactos no se genera	51
14.6.	Carga masiva de contactos falla	51
15.	Tips de Uso - Fase 3	52
15.1.	Optimización	52
15.2.	Seguridad	52
15.3.	Mantenimiento	52

16.Integración Grupal	52
16.1. Conformación de Grupos	52
16.2. Responsabilidades	53
16.3. Manual de Integración	53
17.Conclusión	53
18.Soporte	54

1. Introducción

EDDMail Fase 3 representa la culminación del sistema de simulación de correos electrónicos, incorporando tecnologías avanzadas de seguridad, integridad de datos y optimización de almacenamiento. Esta fase integra Blockchain para asegurar la inmutabilidad de los correos enviados, Árboles de Merkle para verificar la integridad de correos privados, compresión LZW para optimizar el almacenamiento, y grafos para visualizar las relaciones entre usuarios y sus contactos.

1.1. Novedades de la Fase 3

- **Blockchain:** Registro inmutable de todos los correos enviados con proof-of-work
- **Árbol de Merkle:** Verificación criptográfica de integridad de correos favoritos
- **Compresión LZW:** Reducción del tamaño de correos para descarga eficiente
- **Control de Logueo:** Seguimiento detallado de sesiones de usuario
- **Grafos de Contactos:** Visualización de relaciones usuario-contacto (Integración Grupal)
- **Reacciones a Mensajes:** Sistema de interacción en comunidades

1.2. Características Principales

- Todas las funcionalidades de Fase 1 y Fase 2 mantenidas
- Seguridad mediante blockchain con minería de bloques
- Verificación de integridad mediante hashes SHA-256
- Descarga optimizada de correos con compresión LZW
- Sistema de auditoría con control de logueo
- Visualización de redes de contactos mediante grafos
- Exportación de datos en formato JSON
- Carga masiva de contactos (Integración Grupal)

2. Requisitos del Sistema

2.1. Requisitos de Hardware

- Procesador: 1.5 GHz o superior (recomendado 2 GHz para minería blockchain)
- Memoria RAM: 2 GB mínimo (recomendado 4 GB)
- Espacio en disco: 200 MB libres
- Resolución de pantalla: 1024x768 o superior

2.2. Requisitos de Software

- Sistema operativo: Linux (cualquier distribución)
- Free Pascal Compiler (FPC) versión 3.0 o superior
- Lazarus IDE (recomendado versión 2.0+)
- Librería GTK2 instalada
- Graphviz para generación de reportes

2.3. Instalación de Dependencias

Para Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install fpc lazarus-ide
sudo apt-get install libgtk2.0-dev
sudo apt-get install graphviz
```

3. Instalación y Configuración

3.1. Obtener el Código Fuente

1. Clonar el repositorio desde GitHub
2. Navegar al directorio del proyecto Fase 3
3. Abrir el archivo EDDMail.lpr en Lazarus

3.2. Compilación

1. Abrir Lazarus IDE
2. Cargar el proyecto EDDMail.lpr
3. Configurar las dependencias GTK si es necesario
4. Compilar usando F9 o el menú Ejecutar ¿Compilar
5. Verificar que se creen los directorios de reportes automáticamente

4. Capturas de Pantalla del Sistema

Instrucciones: Coloca tus capturas de pantalla en una carpeta llamada imágenes.^{en} el mismo directorio que este archivo .tex. Asegúrate de nombrar los archivos exactamente como se indica a continuación.

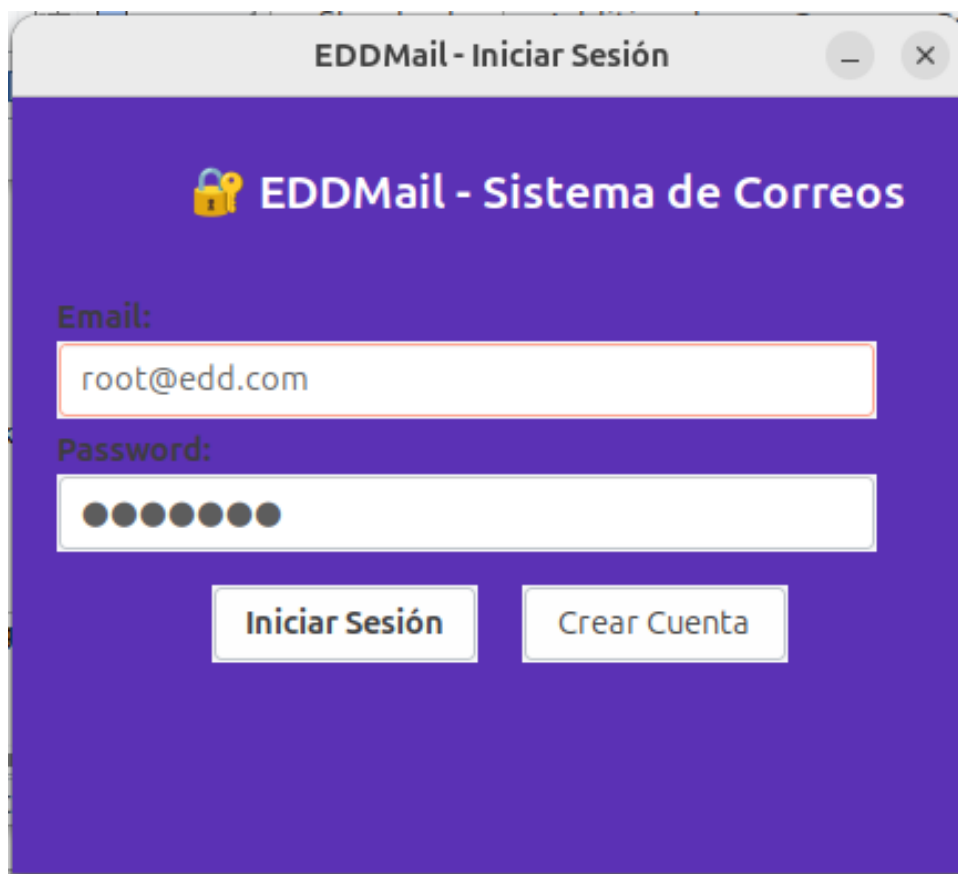


Figura 1: Pantalla de inicio de sesión

A screenshot of a web browser window titled "Crear Nueva Cuenta". The window has a purple background. It contains five input fields with labels: "Nombre completo:", "Nombre de usuario:", "Email:", "Teléfono:", and "Password:". Each label is in bold black text. Below the input fields are two buttons: "Registrar" and "Cancelar". The window has standard browser window controls (minimize, maximize, close) in the top right corner.

Figura 2: Formulario para crear nueva cuenta

- 4.1. Pantalla de Inicio de Sesión
- 4.2. Creación de Cuenta
- 4.3. Panel de Administración (Root)
- 4.4. Bandeja de Entrada
- 4.5. Borradores (Árbol AVL)
- 4.6. Favoritos (Árbol B)
- 4.7. Comunidades (Árbol BST)
- 4.8. Otras Funcionalidades
- 4.9. Reportes del Sistema
- 4.10. Nuevas Capturas - Fase 3
 - 4.10.1. Visualización de Blockchain
 - 4.10.2. Verificación de Árbol de Merkle
 - 4.10.3. Compresión LZW en Correos⁸
 - 4.10.4. Control de Logueo



Figura 3: Panel de administración del usuario root

- **Iniciar Sesión:** Autenticarse con credenciales existentes (se registra en control de logueo)
- **Crear Cuenta:** Registrar un nuevo usuario en el sistema

5.2. Creación de Nueva Cuenta

Para registrar un nuevo usuario (ver Figura 2), complete los siguientes campos:

- Nombre completo
- Nombre de usuario
- Dirección de correo electrónico (debe ser única)
- Número de teléfono
- Contraseña



Figura 4: Interfaz de la bandeja de entrada

6. Usuario Root (Administrador)

El sistema incluye un usuario administrador predefinido:

- **Email:** root@edd.com
- **Password:** root123

6.1. Panel de Administración

El panel de administración (Figura 3) permite al usuario root:

6.1.1. Carga Masiva de Usuarios

Formato JSON:

```
{
  "usuarios": [
    {
      "id": 1,
      "nombre": "Luis Garcia",
      "usuario": "auxluis",
      "email": "aux-luis@edd.com",
      "password": "auxluis123",
      "telefono": "12345678"
    }
  ]
}
```



Figura 5: Formulario para guardar borrador

6.1.2. Carga Masiva de Correos

Formato JSON:

```
{
  "correos": [
    {
      "id": 1,
      "remitente": "aux-luis@edd.com",
      "destinatario": "aux-marcosg@edd.com",
      "estado": "NL",
      "asunto": "Reunion de proyecto",
      "mensaje": "Confirmar asistencia."
    }
  ]
}
```

6.1.3. Carga Masiva de Contactos (NUEVO - Fase 3)

Formato JSON para integración grupal:

```
{
  "usuarios": [
    {
      "email": "usuario1@edd.com",
      "contactos": [
        "contacto1@edd.com",
        "contacto2@edd.com",
        "contacto3@edd.com"
      ]
    },
    {

```

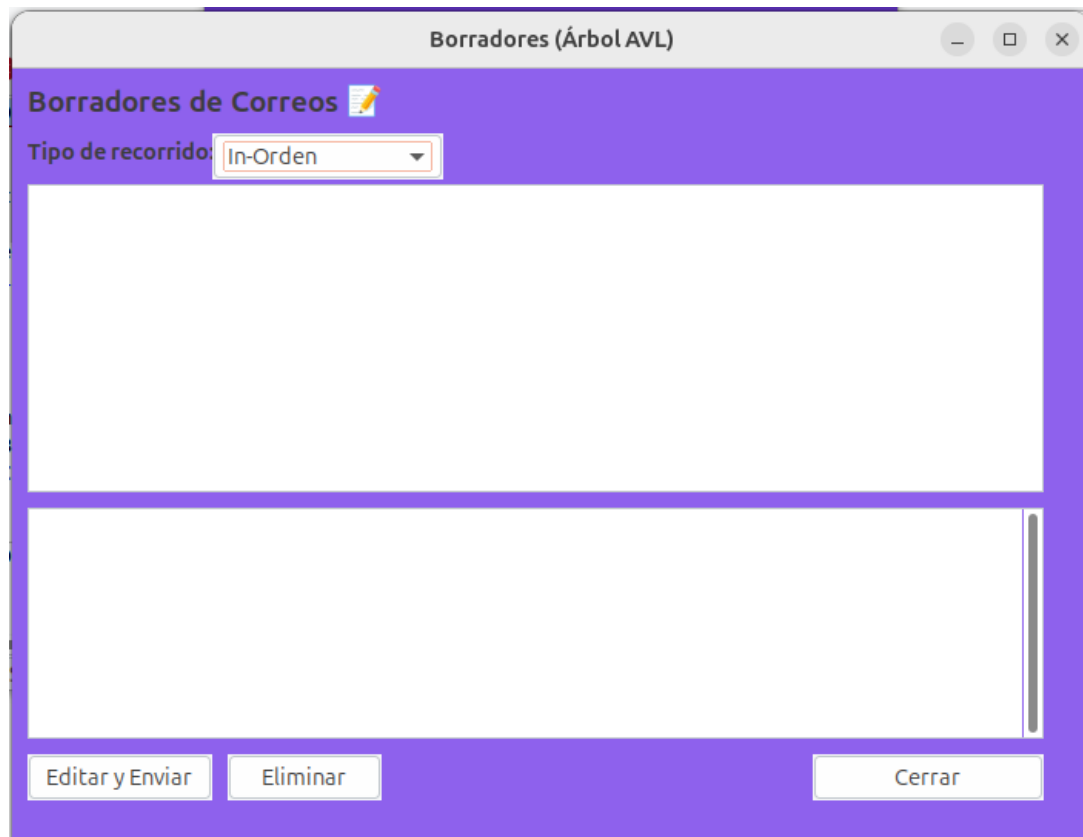


Figura 6: Interfaz para visualizar borradores con recorridos

```

    "email": "usuario2@edd.com",
    "contactos": [
      "contacto4@edd.com",
      "contacto5@edd.com"
    ]
  }
]
}

```

Proceso de carga masiva de contactos:

1. Seleccionar "Carga Masiva de Contactos"
2. Elegir archivo JSON con formato especificado
3. Sistema valida existencia de usuarios
4. Agrega relaciones al grafo de contactos
5. Actualiza lista circular de cada usuario
6. Confirma carga exitosa

6.1.4. Gestión de Comunidades (Árbol BST)

El usuario root puede:

- Crear nuevas comunidades organizadas en árbol BST



Figura 7: Interfaz para marcar correo como favorito

- Asignar usuarios a comunidades existentes
- Visualizar todas las comunidades ordenadas alfabéticamente
- Ver mensajes publicados en cada comunidad
- Generar reportes del árbol BST de comunidades (Figura 18)

6.2. Reportes del Administrador

- **Reporte de Usuarios:** Lista simple de todos los usuarios (Figura 17)
- **Reporte de Relaciones:** Matriz dispersa de correos
- **Reporte de Comunidades BST:** Árbol binario de búsqueda (Figura 18)
- **NUEVO - Reporte de Blockchain:** Cadena completa de bloques minados
- **NUEVO - Reporte de Control de Logueo:** Registro de sesiones
- **NUEVO - Reporte de Grafo de Contactos:** Visualización de red de usuarios

7. Usuario Estándar

7.1. Bandeja de Entrada

La interfaz de bandeja de entrada (Figura 4) muestra una lista doblemente enlazada con correos recibidos:

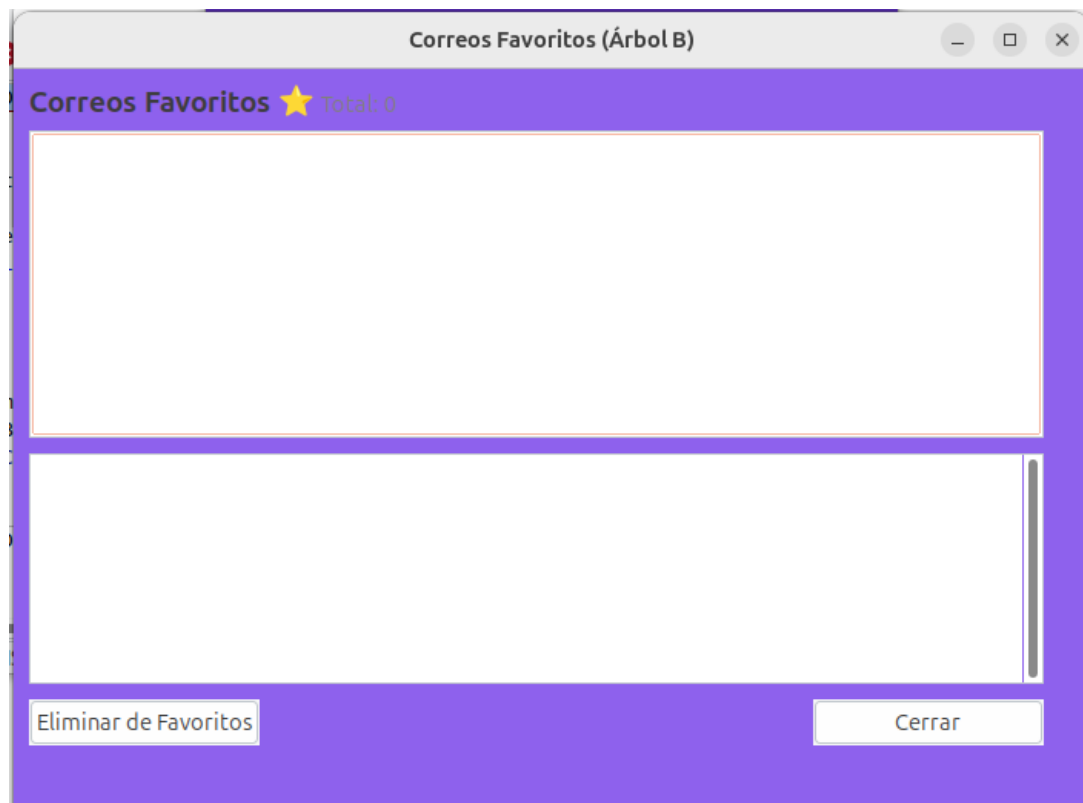


Figura 8: Interfaz de gestión de favoritos

- **Estado:** 'NL' (No Leído) o 'L' (Leído)
- **Asunto:** Título del correo
- **Remitente:** Dirección del emisor
- **Fecha:** Fecha y hora de recepción

Funcionalidades:

- Ordenar por asunto (A-Z)
- Ver correos no leídos
- Marcar como leídos
- Eliminar correos
- Marcar como favoritos (Fase 2)
- Guardar como borrador (Fase 2)
- **NUEVO:** Descargar correo comprimido (Fase 3)

7.2. Borradores (Árbol AVL)

Los borradores se gestionan mediante un Árbol AVL autobalanceado.



Figura 9: Interfaz de comunidades (Árbol BST)

7.2.1. Guardar Borrador

Como se muestra en la Figura 5:

1. Seleccionar "Guardar Borrador"
2. Completar formulario:
 - Destinatario
 - Asunto
 - Mensaje
3. Sistema asigna ID único
4. Se inserta en árbol AVL con balance automático

7.2.2. Visualizar Borradores

La interfaz (Figura 6) permite tres tipos de recorridos:

- **InOrden:** Orden ascendente por ID
- **PreOrden:** Raíz-Izquierda-Derecha
- **PostOrden:** Izquierda-Derecha-Raíz

El reporte gráfico del árbol AVL se muestra en la Figura 19.



Figura 10: Formulario para publicar en comunidad

7.2.3. Modificar y Enviar

- Buscar borrador por ID
- Modificar destinatario, asunto o mensaje
- Enviar borrador (elimina del AVL y envía)
- Eliminar borrador sin enviar

7.3. Favoritos (Árbol B de Orden 5)

Los correos favoritos se organizan en Árbol B de orden 5.

7.3.1. Marcar como Favorito

Proceso (ver Figura 7):

1. Seleccionar correo de bandeja
2. "Marcar como Favorito"
3. Se indexa en Árbol B por ID
4. División automática si nodo lleno

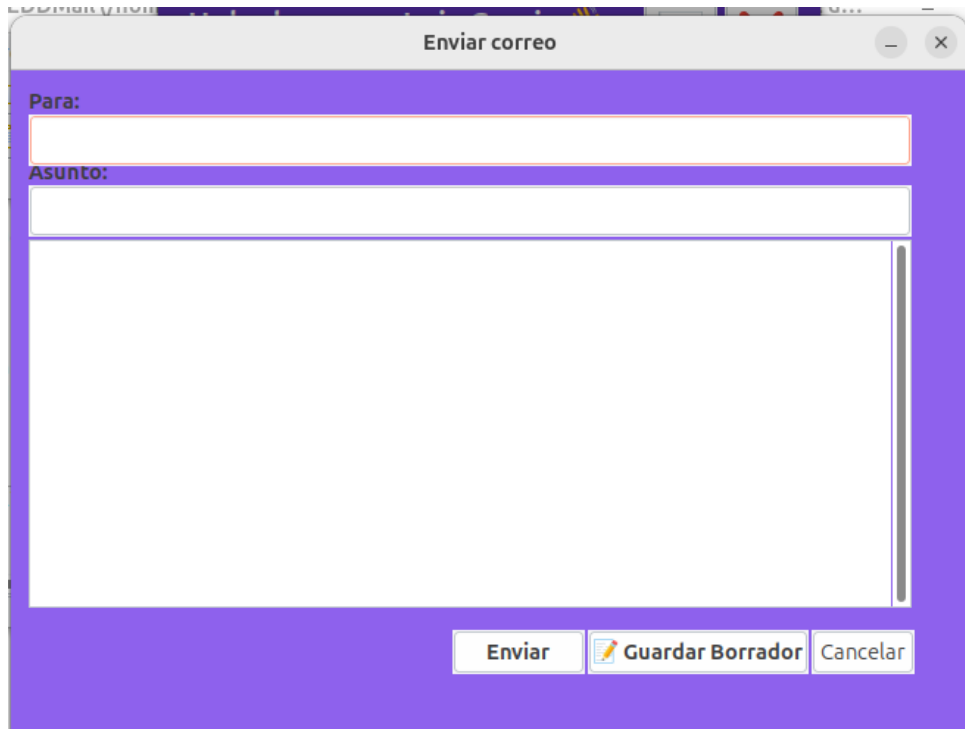


Figura 11: Formulario para enviar correo

7.3.2. Gestionar Favoritos

La interfaz de gestión (Figura 8) permite:

- Ver todos los favoritos
- Buscar por ID (optimizado)
- Desmarcar favoritos
- Generar reporte (Figura 20)

Propiedades del Árbol B:

- Hasta 4 claves por nodo
- Hasta 5 hijos por nodo
- Siempre balanceado
- Búsqueda $O(\log n)$

7.4. Comunidades (Árbol BST)

Sistema de grupos organizados en árbol binario de búsqueda.



Figura 12: Interfaz de navegación de contactos

7.4.1. Unirse a Comunidad

En la interfaz (Figura 9):

1. Ver comunidades disponibles (alfabéticamente)
2. Solicitar unirse
3. Confirmación del sistema

7.4.2. Publicar en Comunidad

Usando el formulario (Figura 10):

1. Acceder a comunidad
2. Redactar mensaje
3. Publicar
4. Se incrementa contador de mensajes

7.4.3. Ver Mensajes

- Listar comunidades (InOrden del BST)
- Seleccionar comunidad
- Ver mensajes cronológicamente
- Mostrar: autor, contenido y fecha

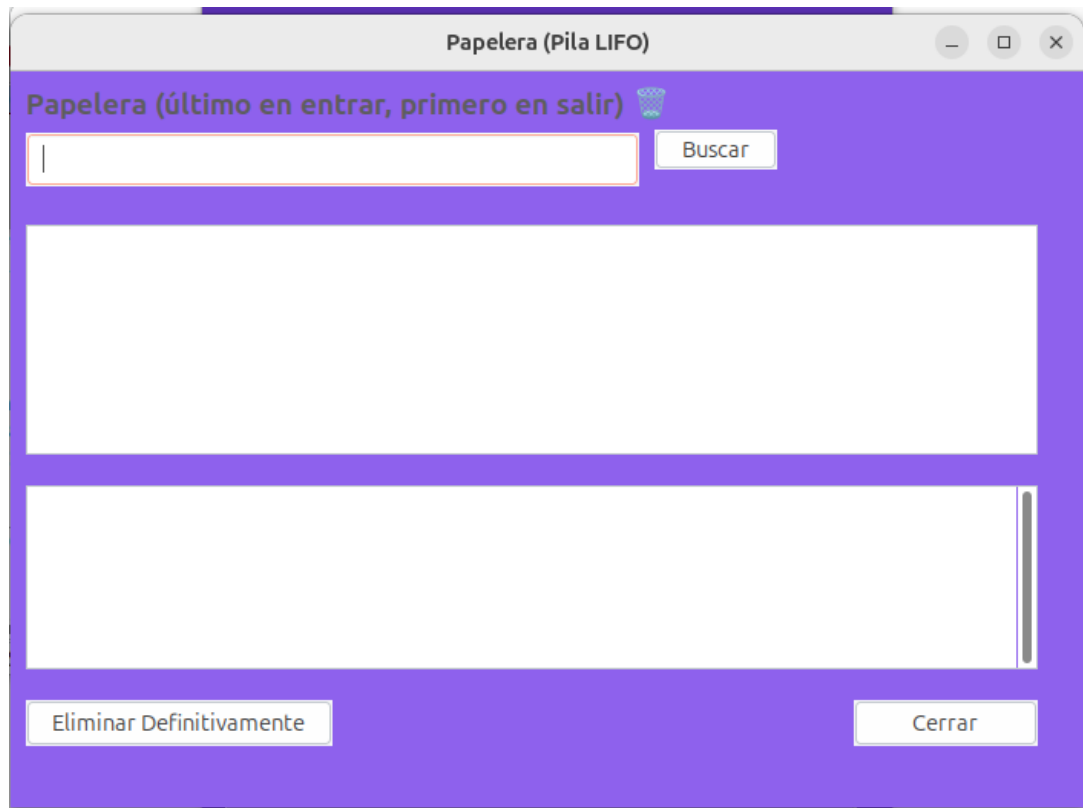


Figura 13: Interfaz de la papelerera (Pila LIFO)

7.4.4. Reaccionar a Mensajes (NUEVO - Fase 3)

Sistema de reacciones estilo "Me gusta":

1. Ingresar a comunidad
2. Ver mensajes publicados con [ID: X]
3. Hacer clic en "Reaccionar"
4. Ingresar ID del mensaje
5. Sistema incrementa contador de reacciones
6. Contador se muestra junto al mensaje

Características:

- Cada mensaje tiene contador de reacciones
- Usuarios pueden reaccionar múltiples veces
- Reacciones se almacenan en estructura del mensaje
- Visualización en tiempo real

Figura 14: Formulario para programar correo

7.5. Otras Funcionalidades

7.5.1. Enviar Correo

El formulario (Figura 11) permite:

- Destinatario (debe estar en contactos)
- Asunto
- Mensaje
- Actualiza matriz dispersa
- **NUEVO:** Registra en blockchain automáticamente

7.5.2. Gestión de Contactos

Como se muestra en la Figura 12:

- Agregar contacto (lista circular)
- Navegar: Anterior/Siguiente
- Ver información completa
- Generar reporte (Figura 23)

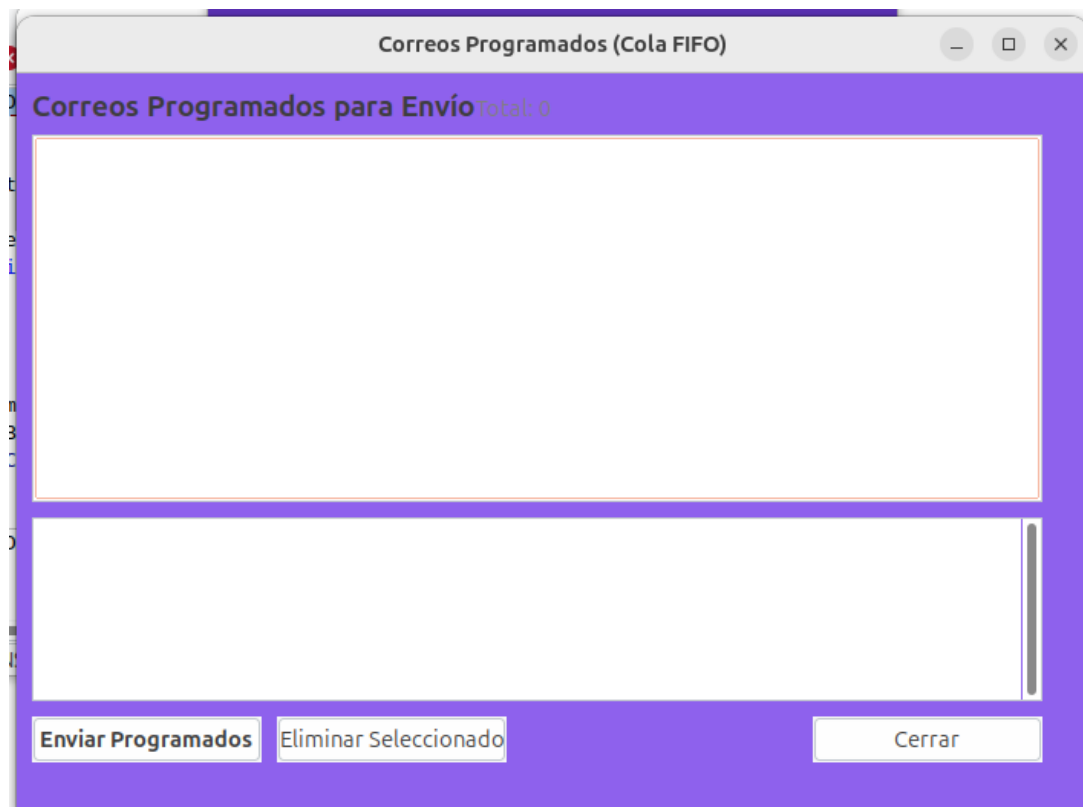


Figura 15: Lista de correos programados (Cola FIFO)

- **NUEVO:** Relaciones se reflejan en grafo

7.5.3. Papelera

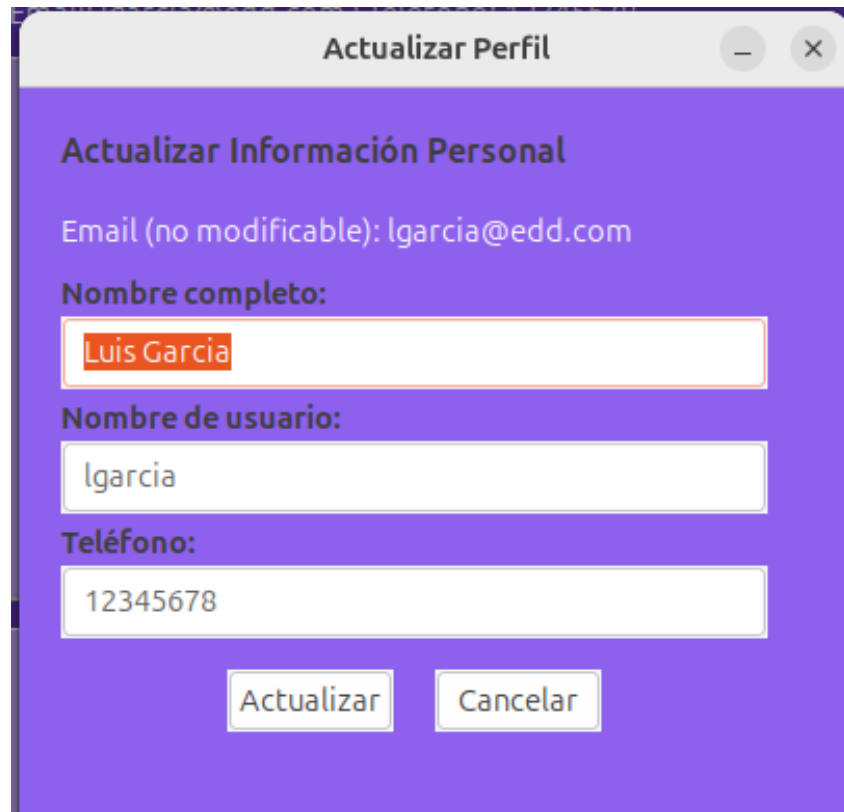
La interfaz (Figura 13) muestra:

- Estructura: Pila LIFO
- Buscar por asunto
- Eliminar definitivamente
- Visualizar correos
- Reporte gráfico (Figura 22)
- **NUEVO:** Vaciar papelera completa

7.5.4. Correos Programados

Funcionalidades (Figuras 14 y 15):

- Estructura: Cola FIFO
- Programar con fecha/hora
- Enviar manualmente
- Eliminar programados



Actualizar Perfil

Actualizar Información Personal

Email (no modificable): lgarcia@edd.com

Nombre completo:

Luis Garcia

Nombre de usuario:

lgarcia

Teléfono:

12345678

Actualizar Cancelar

Figura 16: Formulario para actualizar perfil

7.5.5. Actualizar Perfil

El formulario (Figura 16) permite:

- Modificar nombre
- Modificar usuario
- Modificar teléfono
- Email no modificable

8. Nuevas Funcionalidades - Fase 3

8.1. Blockchain

El sistema implementa una cadena de bloques (blockchain) para registrar todos los correos enviados, garantizando inmutabilidad e integridad mediante proof-of-work.

8.1.1. Arquitectura del Blockchain

Cada bloque contiene:

- **Index:** Número secuencial del bloque (0 para génesis)
- **Timestamp:** Fecha y hora en formato DD-MM-YY::HH:MM:SS

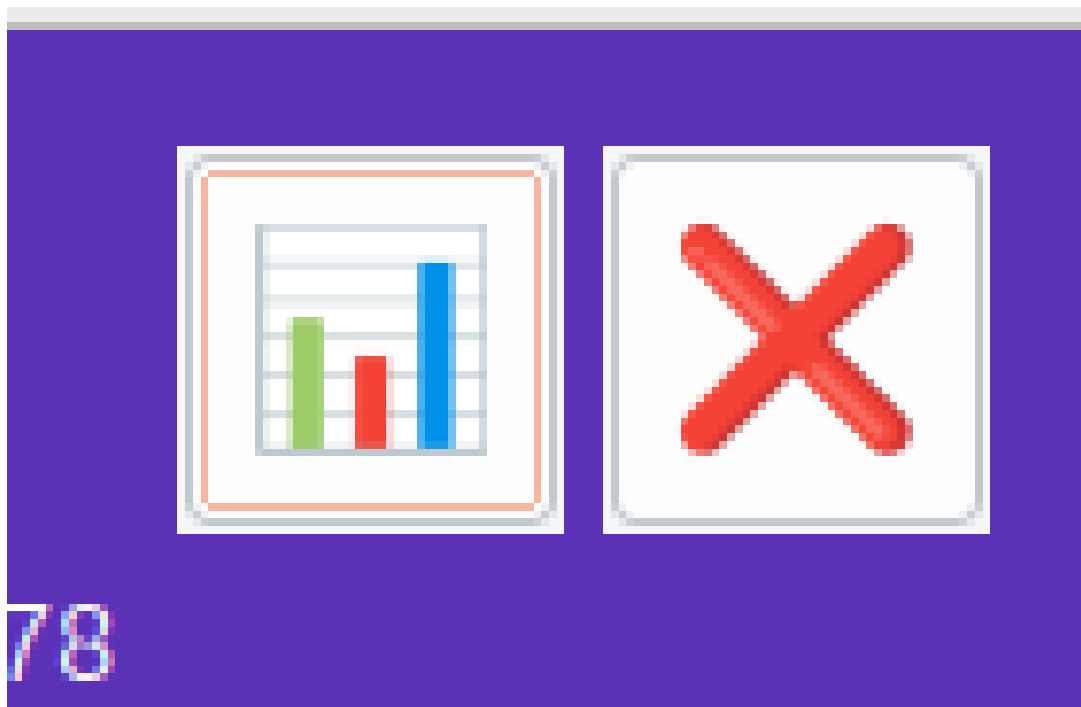


Figura 17: Ejemplo de reporte de usuarios (Lista Simple)

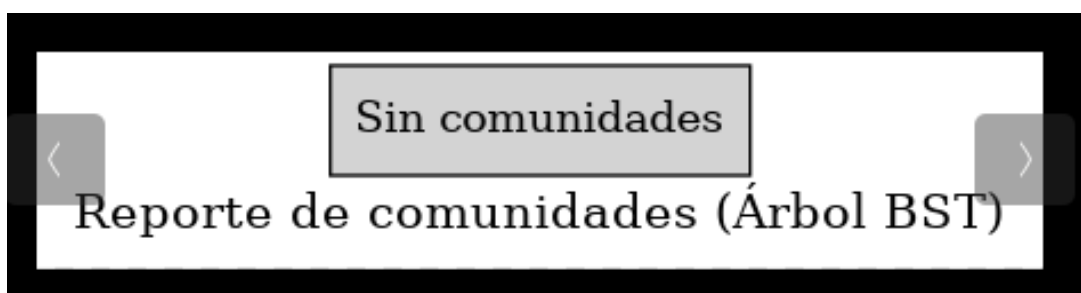


Figura 18: Ejemplo de reporte de comunidades (Árbol BST)

- **Data:** Información del correo (ID, Remitente, Asunto, Mensaje)
- **Nonce:** Número usado para proof-of-work
- **PreviousHash:** Hash del bloque anterior (enlace)
- **Hash:** Hash SHA-256 del bloque actual

8.1.2. Bloque Génesis

Al iniciar el sistema, se crea automáticamente el bloque génesis:

```
Block 0 (Genesis)
Index: 0
Timestamp: 2025-09-30 00:00
Data: Genesis Block
Nonce: 12345
Prev Hash: 0
Hash: abc123...
```

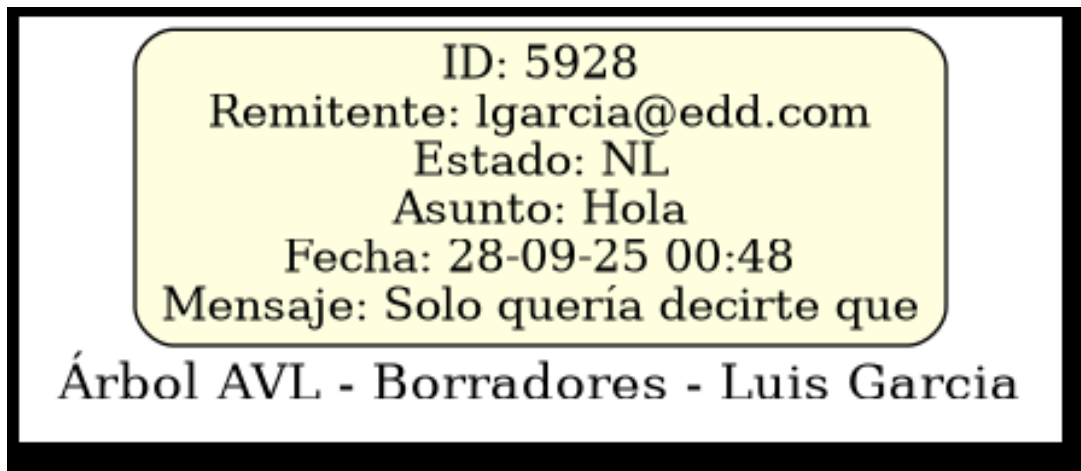



Figura 19: Ejemplo de reporte de borradores (Árbol AVL)

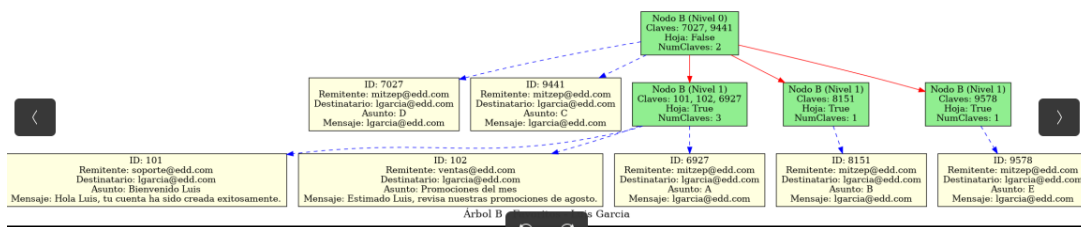


Figura 20: Ejemplo de reporte de favoritos (Árbol B orden 5)

8.1.3. Minería de Bloques

Cuando un usuario envía un correo:

1. Sistema crea nuevo bloque con datos del correo
2. Inicia proceso de minería (proof-of-work)
3. Busca nonce que genere hash con prefijo "000"
4. Valida que hash cumpla dificultad
5. Enlaza bloque a la cadena
6. Correo queda registrado de forma inmutable

Proof-of-Work:

- Hash debe iniciar con "000" (dificultad configurable)
- Sistema incrementa nonce hasta encontrar hash válido
- Garantiza trabajo computacional para añadir bloques
- Previene manipulación de la cadena

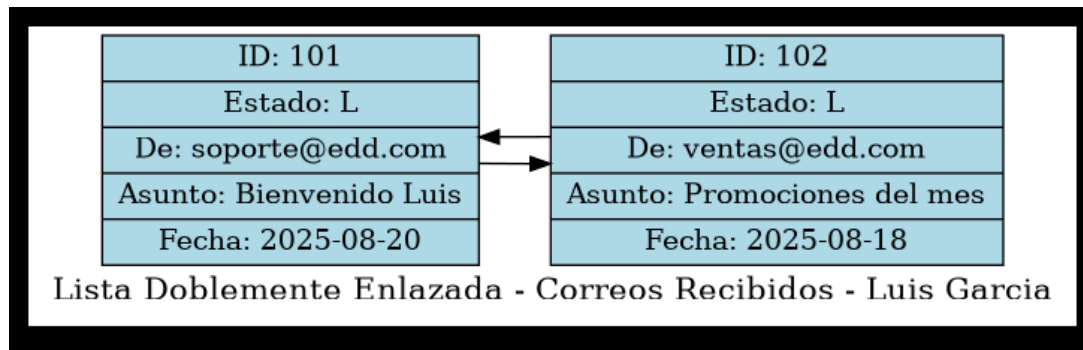


Figura 21: Ejemplo de reporte de bandeja de entrada (Lista Doble)

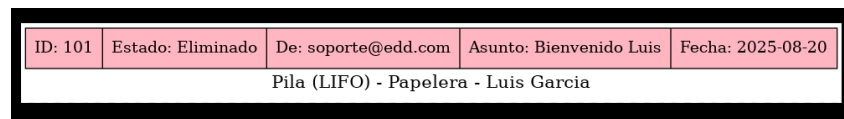


Figura 22: Ejemplo de reporte de papelera (Pila)

8.1.4. Visualización del Blockchain

Interfaz de usuario:

1. Acceder a "Ver Blockchain"
2. Lista todos los bloques en orden cronológico
3. Muestra: Index, Timestamp, Data abreviado
4. Seleccionar bloque para ver detalles completos:
 - Index completo
 - Timestamp detallado
 - Data completa del correo
 - Nonce utilizado
 - Previous Hash
 - Hash del bloque

Verificación de integridad:

- Sistema puede verificar cadena completa
- Recalcula hash de cada bloque
- Valida enlaces entre bloques
- Detecta cualquier alteración



Figura 23: Ejemplo de reporte de contactos (Lista Circular)

8.1.5. Reporte Graphviz

El reporte gráfico del blockchain muestra:

- Bloques en secuencia vertical
- Cada bloque como rectángulo con información:
 - Index
 - Timestamp
 - Data (abreviado)
 - Nonce
 - Previous Hash (primeros 6 caracteres)
 - Hash (primeros 6 caracteres)
- Flechas indicando enlaces Previous Hash → Hash
- Color distintivo para bloque génesis

Ejemplo visual:

```
Block 0 (Genesis)
Index: 0
Timestamp: 2025-09-30
Data: Genesis Block
Nonce: 12345
Prev Hash: 0
Hash: abc123
```

```
Block 1
```

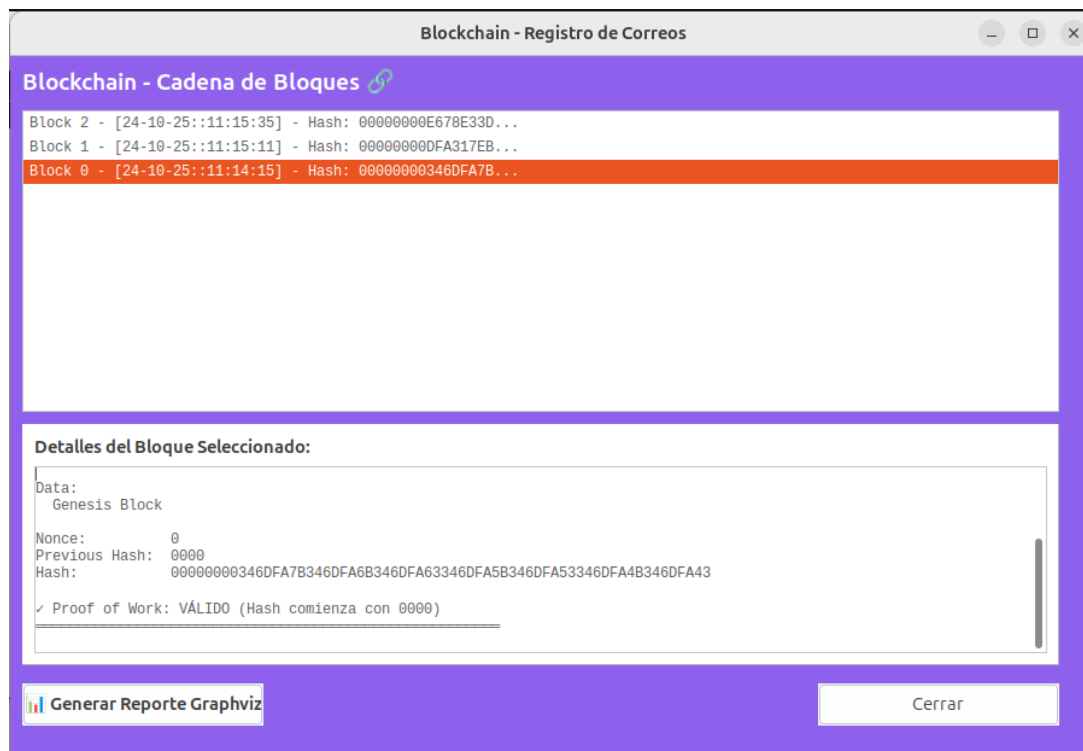


Figura 24: Interfaz de visualización de la cadena de bloques (Blockchain).

Index: 1
 Timestamp: 2025-09-30
 Data: ID: 1, Remit...
 Nonce: 67890
 Prev Hash: abc123
 Hash: def456

8.2. Árbol de Merkle

El Árbol de Merkle garantiza la integridad de los correos favoritos mediante verificación criptográfica.

8.2.1. Construcción del Árbol

Proceso automático:

1. Al marcar correos como favoritos, sistema activa construcción
2. Recolecta todos los correos favoritos del usuario
3. Calcula hash SHA-256 de cada correo (nodo hoja):
 - Concatena: ID + Remitente + Asunto + Fecha + Mensaje
 - Aplica función hash
 - Crea nodo hoja con hash y referencia al correo

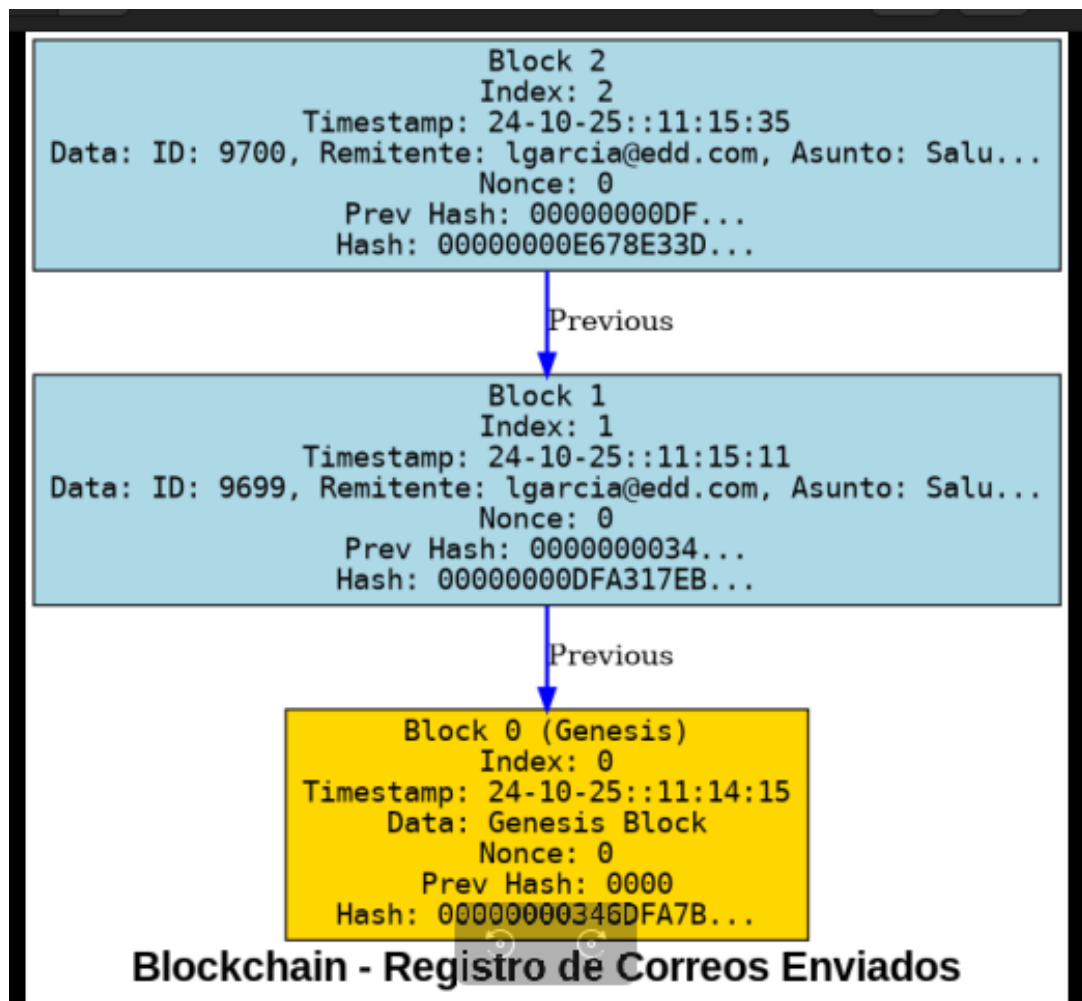


Figura 25: Reporte gráfico del Blockchain generado con Graphviz.

4. Agrupa hojas en pares
5. Combina hashes de pares (concatenar + hash) para crear nodos padres
6. Repite proceso hasta obtener hash raíz único
7. Hash raíz representa integridad de todos los favoritos

8.2.2. Propiedades del Árbol de Merkle

- **Integridad:** Cualquier cambio en un correo modifica hash raíz
- **Eficiencia:** Verificación $O(\log n)$
- **Compacto:** Solo hash raíz necesario para verificar conjunto
- **Pruebas de pertenencia:** Verifica correo sin revisar todos
- **Balance:** Árbol binario balanceado automáticamente



Figura 26: Interfaz de usuario para verificación de integridad mediante Árbol de Merkle.

8.2.3. Nodos del Árbol

Nodos hoja (hojas):

- Contienen puntero al correo original
- Hash de datos completos del correo
- Marca EsHoja = True
- Sin hijos

Nodos internos:

- No contienen correos
- Hash = combinación de hashes de hijos

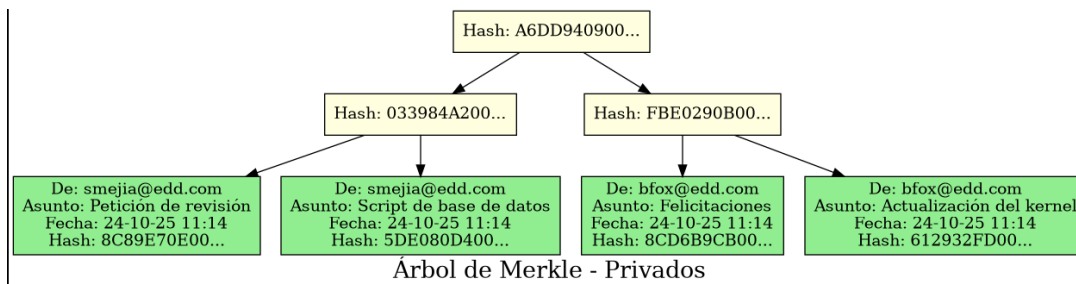


Figura 27: Reporte visual del Árbol de Merkle con nodos hoja y raíz hash.

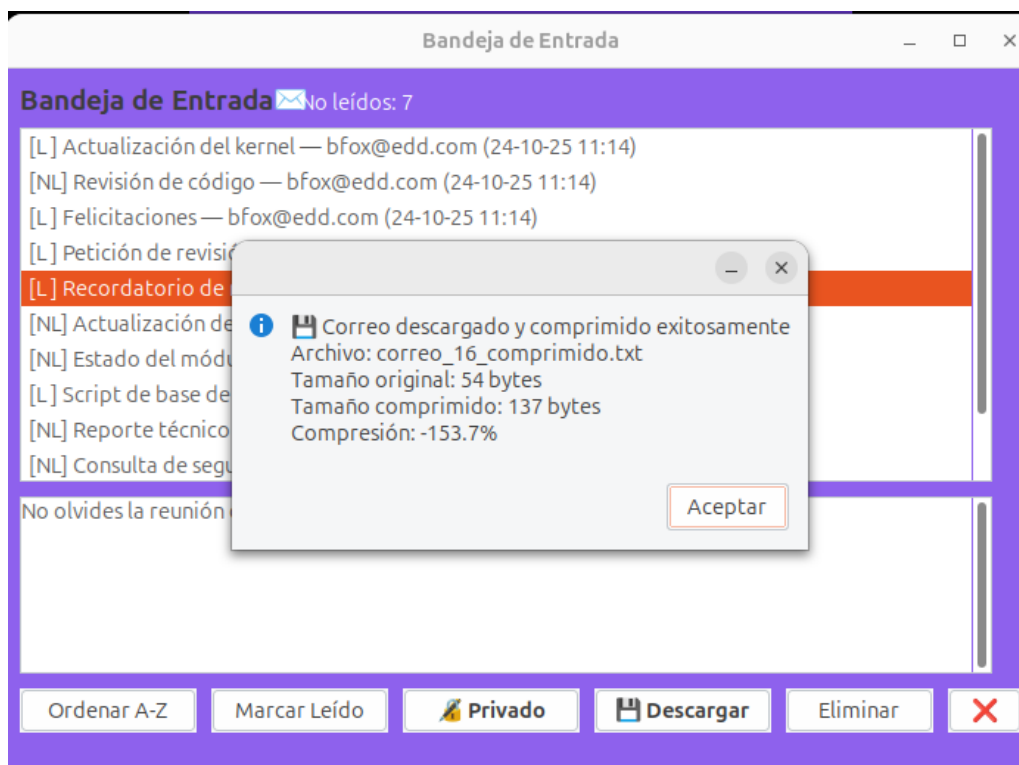


Figura 28: Interfaz de descarga de correo comprimido (LZW).

- Dos hijos (izquierdo y derecho)
- Marca EsHoja = False

Nodo raíz:

- Hash representa integridad total
- Almacenado en estructura del usuario
- Punto de verificación principal

8.2.4. Verificación de Integridad

Para verificar un correo favorito:

1. Localizar hoja correspondiente



Figura 29: Ejemplo del archivo comprimido generado con información de reducción de tamaño.

2. Recalcular hash del correo actual
3. Comparar con hash almacenado en hoja
4. Si difieren, correo fue modificado
5. Recalcular hash raíz desde esa hoja
6. Comparar con hash raíz almacenado
7. Detectar exactamente qué correo cambió

8.2.5. Búsqueda en Árbol de Merkle

Nueva funcionalidad de Fase 3:

1. Acceder a "Buscar en Favoritos (Merkle)"
2. Ingresar ID del correo
3. Sistema busca en árbol de Merkle:
 - Recorre árbol hasta encontrar hoja con ese correo
 - Verifica integridad del correo
 - Muestra datos completos del correo
 - Indica si hash es válido
4. Complejidad: $O(\log n)$ vs $O(n)$ en lista

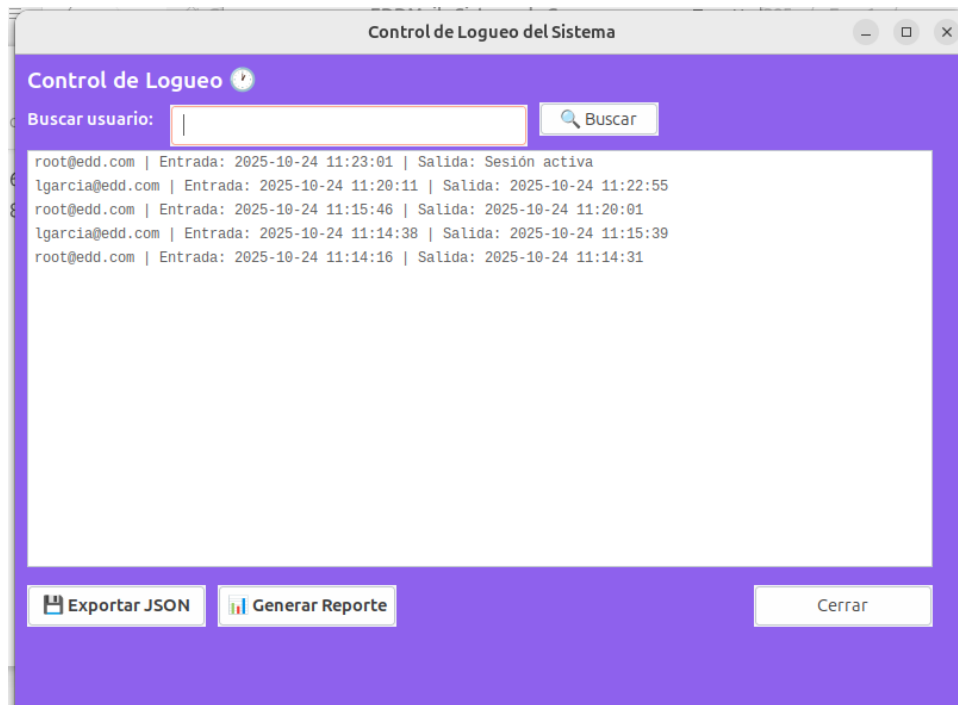


Figura 30: Interfaz de control de logueo mostrando registros de entrada y salida.

8.2.6. Reporte Graphviz

El reporte visual del Árbol de Merkle muestra:

- Estructura jerárquica del árbol
- **Nodo raíz** (arriba):
 - Etiqueta "Privados"
 - Hash completo (64 caracteres)
- **Nodos internos**:
 - Hash combinado (primeros 10 caracteres mostrados)
 - Conexiones a hijos
- **Nodos hoja** (abajo):
 - Información del correo: De, Asunto, Fecha
 - Hash del correo (primeros 10 caracteres)
 - Color distintivo (verde claro)

Ejemplo visual:

```

Privados
Hash: 417aad8a...
```

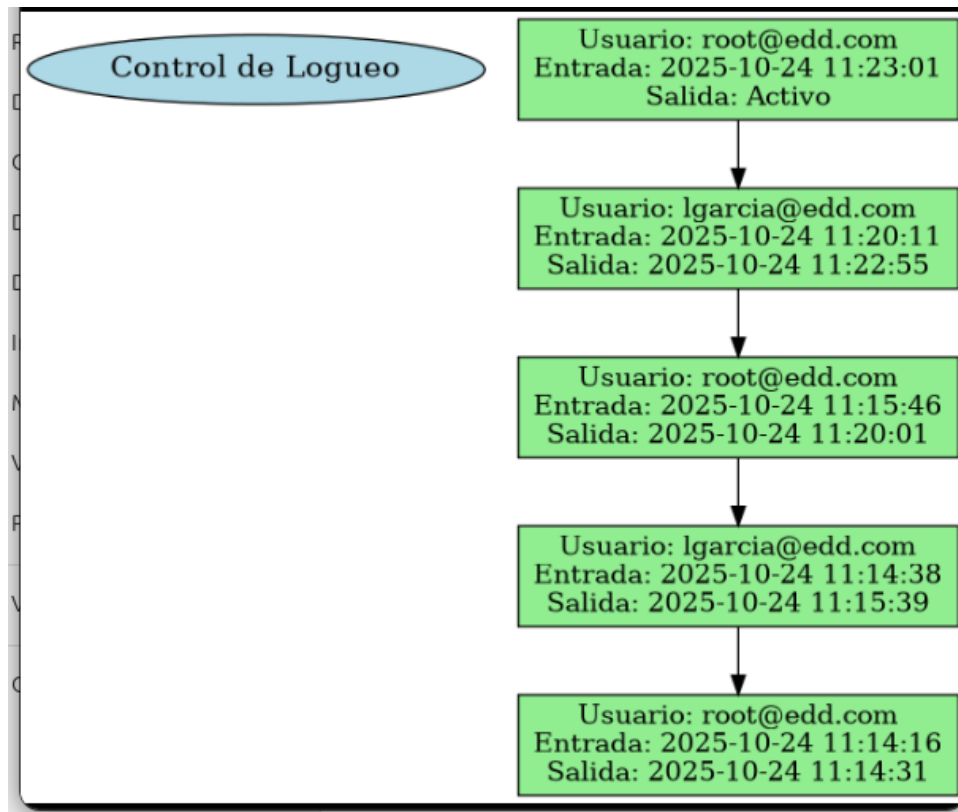


Figura 31: Reporte Graphviz de los registros de logueo.

Hash: 3c1a6ef7... Hash: 81b77e45...

Hoja	Hoja	Hoja
ID:1	ID:2	ID:3

8.3. Compresión LZW

Implementación del algoritmo Lempel-Ziv-Welch para comprimir correos antes de descargarlos.

8.3.1. Algoritmo LZW

Funcionamiento:

1. Inicializar diccionario con caracteres individuales (0-255)
2. Leer entrada carácter por carácter

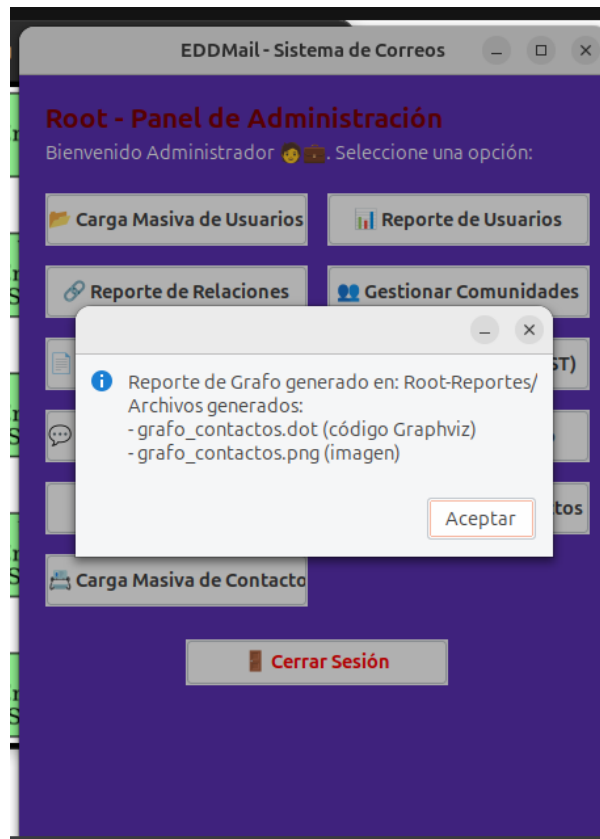


Figura 32: Interfaz de carga masiva de contactos y visualización del grafo.

3. Buscar secuencia más larga en diccionario
4. Emitir código de esa secuencia
5. Agregar nueva secuencia al diccionario
6. Repetir hasta procesar todo el texto

Ventajas:

- Sin pérdida de información
- Compresión adaptativa (aprende patrones)
- Eficiente para textos repetitivos
- Descompresión rápida
- No requiere diccionario previo

8.3.2. Descargar Correo Comprimido

Proceso del usuario:

1. Seleccionar correo en bandeja de entrada
2. Hacer clic en " Descargar Comprimido"



Figura 33: Reporte del grafo de contactos generado con Graphviz.

3. Sistema ejecuta compresión:

- Extrae contenido completo del correo
- Aplica algoritmo LZW
- Genera archivo de texto comprimido

4. Aparece diálogo "Guardar como"

5. Usuario selecciona ubicación

6. Archivo guardado con extensión .txt

7. Confirmación de descarga exitosa

8.3.3. Formato del Archivo Comprimido

El archivo descargado contiene:

```
=== CORREO COMPRIMIDO - LZW ===  
ID: 42  
Remitente: usuario@edd.com  
Destinatario: destino@edd.com  
Asunto: Reuni n importante  
Fecha: 2025-09-30 14:30  
  
=== MENSAJE COMPRIMIDO ===  
<secuencia de c dgos LZW>  
72,101,108,108,111,256,119,111,114,108,100...  
  
=== INFORMACI N ===  
Tama o original: 350 bytes  
Tama o comprimido: 187 bytes  
Ratio de compresi n: 46.57%
```

8.3.4. Tasas de Compresi3n

Rendimiento esperado:

- Textos con mucha repetici3n: 40-60 % reducci3n
- Correos formales: 25-40 % reducci3n
- Mensajes cortos: 10-25 % reducci3n
- Textos aleatorios: 0-10 % reducci3n

Factores que mejoran compresi3n:

- Palabras repetidas frecuentemente
- Frases comunes (saludos, despedidas)
- Formato estructurado
- Longitud del mensaje (mejora con tama1o)

8.3.5. Casos de Uso

- **Respaldo:** Guardar correos importantes ocupando menos espacio
- **Transferencia:** Enviar correos por medios limitados
- **Archivado:** Mantener hist3rico sin consumir mucho disco
- **An1lisis:** Estudiar eficiencia de compresi3n en comunicaciones

8.4. Control de Logueo

Sistema de auditoría que registra todas las sesiones de usuario.

8.4.1. Estructura del Registro

Cada registro de logueo contiene:

- **Usuario:** Email del usuario
- **Entrada:** Timestamp de inicio de sesión (TDateTime)
- **Salida:** Timestamp de cierre de sesión (TDateTime)
- **Siguiente:** Enlace al siguiente registro (lista enlazada)

8.4.2. Registro Automático

Al iniciar sesión:

1. Usuario ingresa credenciales
2. Sistema valida usuario y contraseña
3. Si autenticación exitosa:
 - Crea nuevo registro de logueo
 - Establece timestamp de entrada (hora actual)
 - Campo Salida queda pendiente
 - Añade registro a lista de logueos
4. Usuario accede al sistema

Al cerrar sesión:

1. Usuario selecciona "Cerrar Sesión"
2. Sistema localiza registro activo del usuario
3. Establece timestamp de salida (hora actual)
4. Calcula duración de sesión
5. Cierra sesión y retorna a login

8.4.3. Visualización del Control de Logueo

Interfaz "Ver Control de Logueo" (solo Root):

1. Acceder desde panel administrativo
2. Se abre ventana con lista de registros
3. Muestra todos los logueos en formato:

Usuario: usuario@edd.com
Entrada: 2025-09-30 08:15:30
Salida: 2025-09-30 10:45:22
Duración: 2h 29m 52s

4. Funcionalidades disponibles:

- **Buscar:** Filtrar por usuario específico
- **Exportar JSON:** Guardar registros
- **Generar Reporte:** Crear gráfico Graphviz

8.4.4. Búsqueda en Logueo

1. Ingresar nombre de usuario en campo de búsqueda
2. Hacer clic en "Buscar"
3. Sistema filtra registros:
 - Solo muestra logueos de ese usuario
 - Ordenados cronológicamente
 - Incluye todas las sesiones históricas
4. Dejar campo vacío y buscar para ver todos

8.4.5. Exportación JSON

Formato del archivo exportado:

```
{
  "logueos": [
    {
      "usuario": "usuario1@edd.com",
      "entrada": "2025-09-30T08:15:30",
      "salida": "2025-09-30T10:45:22",
      "duracion": "2:29:52"
    },
    {
      "usuario": "usuario2@edd.com",
      "entrada": "2025-09-30T09:00:15",
      "salida": "2025-09-30T09:35:40",
      "duracion": "0:35:25"
    }
  ],
  "total_registros": 2,
  "fecha_exportacion": "2025-10-01T14:30:00"
}
```

8.4.6. Reporte Graphviz

El reporte gráfico muestra:

- Lista secuencial de registros
- Cada nodo contiene:
 - Usuario
 - Hora de entrada
 - Hora de salida
 - Duración calculada
- Flechas indicando secuencia temporal
- Color diferenciado por usuario
- Formato tabular para fácil lectura

8.4.7. Casos de Uso

- **Auditoría:** Rastrear accesos al sistema
- **Seguridad:** Detectar accesos no autorizados
- **Estadísticas:** Analizar patrones de uso
- **Compliance:** Cumplir requisitos de registro
- **Análisis:** Tiempo promedio de sesiones

8.5. Grafo de Contactos (Integración Grupal)

Visualización de relaciones usuario-contacto mediante grafos no dirigidos.

8.5.1. Estructura del Grafo

Características:

- Grafo no dirigido: relación bidireccional
- Nodos representan usuarios
- Aristas representan relación de contacto
- Cada nodo almacena:
 - ID único
 - Nombre completo
 - Usuario (username)
 - Email

8.5.2. Construcción del Grafo

El grafo se construye automáticamente:

1. Al agregar contacto manualmente:
 - Usuario agrega contacto desde su interfaz
 - Sistema crea/actualiza nodo del usuario
 - Crea/actualiza nodo del contacto
 - Establece arista bidireccional
2. Al cargar contactos desde JSON (carga masiva):
 - Root carga archivo JSON
 - Sistema parsea usuarios y contactos
 - Crea todos los nodos
 - Establece todas las aristas
 - Valida que usuarios existan en sistema

8.5.3. Carga Masiva de Contactos

Funcionalidad grupal (3 integrantes):

1. Root accede a "Carga Masiva de Contactos"
2. Selecciona archivo JSON con formato:

```
{
  "usuarios": [
    {
      "email": "usuario1@edd.com",
      "contactos": [
        "auxmarcos@edd.com",
        "auxsteven@edd.com"
      ]
    },
    {
      "email": "usuario2@edd.com",
      "contactos": [
        "auxluis@edd.com"
      ]
    }
  ]
}
```

3. Sistema procesa archivo:
 - Valida formato JSON
 - Verifica existencia de usuarios
 - Agrega contactos a lista circular de cada usuario
 - Actualiza grafo de contactos
 - Reporta éxitos y errores
4. Confirmación de carga completa

8.5.4. Visualización del Grafo

Reporte Graphviz:

- Nodos circulares representan usuarios
- Tamaño proporcional a número de contactos
- Color distintivo:
 - Azul claro: usuarios regulares
 - Verde claro: sus contactos
- Aristas sin dirección (líneas simples)
- Etiquetas con información:
 - ID
 - Usuario (username)
 - Número de contactos

Ejemplo visual:

```
ID: 1
Usuario
Luis
```

```
ID: 3  ID: 4
Marcos  Steven
```

```
ID: 5
Luis
```

8.5.5. Análisis del Grafo

Información que se puede obtener:

- **Grado de cada nodo:** Número de contactos
- **Componentes conexas:** Grupos aislados
- **Nodos centrales:** Usuarios con más conexiones
- **Caminos:** Relaciones indirectas entre usuarios
- **Densidad:** Qué tan conectada está la red

8.5.6. Generación del Reporte

1. Root selecciona Reporte de Grafo de Contactos”
2. Sistema recorre estructura de contactos:
 - Identifica todos los usuarios
 - Mapea sus contactos
 - Genera código DOT para Graphviz
3. Ejecuta Graphviz para crear imagen PNG
4. Guarda en carpeta Root-Reportes/
5. Muestra confirmación con ruta del archivo

9. Estructuras de Datos

9.1. Fase 1 (Mantenidas)

- **Lista Simple:** Usuarios
- **Lista Doblemente Enlazada:** Bandeja entrada
- **Lista Circular:** Contactos
- **Pila LIFO:** Papelera
- **Cola FIFO:** Programados
- **Matriz Dispersa:** Relaciones

9.2. Fase 2 (Mantenidas)

- **Árbol AVL:** Borradores
- **Árbol BST:** Comunidades
- **Árbol B Orden 5:** Favoritos (migrado a Merkle)

9.3. Fase 3 (Nuevas)

9.3.1. Blockchain

Uso: Registro inmutable de correos enviados

Características:

- Cadena enlazada de bloques
- Cada bloque contiene hash del anterior
- Proof-of-work para minería

- Validación de integridad
- Inmutabilidad garantizada
- Complejidad inserción: $O(n)$ por minería
- Complejidad verificación: $O(n)$

9.3.2. Árbol de Merkle

Uso: Verificación de integridad de favoritos

Características:

- Árbol binario balanceado
- Hojas contienen hashes de correos
- Nodos internos: hash combinado de hijos
- Raíz: hash de todos los favoritos
- Verificación eficiente: $O(\log n)$
- Detección de cambios: $O(\log n)$
- Compacto: solo raíz necesaria

9.3.3. Grafo No Dirigido

Uso: Relaciones usuario-contacto

Características:

- Nodos: usuarios del sistema
- Aristas: relaciones de contacto
- No dirigido: relación bidireccional
- Búsqueda: $O(V + E)$
- Inserción nodo: $O(1)$
- Inserción arista: $O(1)$
- Recorrido: BFS o DFS

9.3.4. Lista Enlazada Simple (Logueo)

Uso: Control de sesiones

Características:

- Registro cronológico
- Cada nodo: entrada + salida
- Inserción al final: $O(1)$
- Búsqueda: $O(n)$
- No requiere orden especial

10. Tabla de Complejidades

Estructura	Búsqueda	Inserción	Eliminación
Lista Simple	$O(n)$	$O(1)$	$O(n)$
Lista Doble	$O(n)$	$O(1)$	$O(n)$
Lista Circular	$O(n)$	$O(1)$	$O(n)$
Pila	$O(n)$	$O(1)$	$O(1)$
Cola	$O(n)$	$O(1)$	$O(1)$
Árbol AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$
Árbol BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
Árbol B	$O(\log n)$	$O(\log n)$	$O(\log n)$
Matriz Dispersa	$O(m+n)$	$O(1)$	$O(1)$
Blockchain	$O(n)$	$O(n)^*$	N/A
Merkle Tree	$O(\log n)$	$O(n)$	$O(n)$
Grafo	$O(V+E)$	$O(1)$	$O(V+E)$
Logueo	$O(n)$	$O(1)$	N/A

* Incluye tiempo de minería (proof-of-work)

11. Sistema de Reportes

11.1. Reportes Root

Carpeta: Root-Reportes/

- Usuarios (Lista Simple) - Figura 17
- Relaciones (Matriz Dispersa)
- Comunidades (Árbol BST) - Figura 18
- **NUEVO:** Blockchain (Cadena de bloques)
- **NUEVO:** Control de Logueo (Lista registros)
- **NUEVO:** Grafo de Contactos (Grafo no dirigido)

11.2. Reportes Usuario

Carpeta: [USUARIO]-Reportes/

- Bandeja (Lista Doble) - Figura 21
- Papelera (Pila) - Figura 22
- Programados (Cola)
- Contactos (Lista Circular) - Figura 23
- Borradores (Árbol AVL) - Figura 19
- Favoritos (Árbol B) - Figura 20
- **NUEVO:** Merkle Tree (Árbol de Merkle)

12. Ejemplos de Uso - Fase 3

12.1. Ejemplo 1: Enviar Correo con Blockchain

1. Login: usuario@edd.com
2. Ir a ".Enviar Correo"
3. Completar:
 - Destinatario: destino@edd.com
 - Asunto: Proyecto Fase 3
 - Mensaje: Implementación completa
4. Hacer clic en ".Enviar"
5. Sistema automáticamente:
 - Crea bloque con datos del correo
 - Inicia minería (busca nonce válido)
 - Hash debe iniciar con "000"
 - Enlaza a blockchain
 - Confirma envío
6. Correo enviado y registrado inmutablemente

12.2. Ejemplo 2: Verificar Integridad con Merkle

1. Usuario tiene 5 correos favoritos
2. Sistema construyó Árbol de Merkle
3. Acceder a "Ver Favoritos"
4. Buscar correo por ID: 42
5. Sistema:
 - Busca en árbol: $O(\log n)$
 - Encuentra hoja correspondiente
 - Recalcula hash del correo
 - Compara con hash almacenado
 - Muestra "Integridad verificada"
6. Generar reporte para visualizar árbol
7. Ver estructura jerárquica con hashes

12.3. Ejemplo 3: Descargar Correo Comprimido

1. Usuario en bandeja de entrada
2. Selecciona correo ID: 15
3. Clic en " Descargar Comprimido"
4. Sistema ejecuta LZW:
 - Extrae texto completo
 - Aplica compresión
 - Tamaño original: 450 bytes
 - Tamaño comprimido: 234 bytes
 - Ratio: 48 % reducción
5. Aparece diálogo guardar
6. Seleccionar ubicación
7. Archivo guardado: correo_15_comprimido.txt
8. Mensaje: " Descarga exitosa"

12.4. Ejemplo 4: Auditoría con Control de Logueo

1. Root hace login
2. Accede a "Ver Control de Logueo"
3. Ve todos los registros:

```
usuario1@edd.com - 08:15 - 10:45 (2h 30m)
usuario2@edd.com - 09:00 - 09:35 (0h 35m)
usuario1@edd.com - 14:20 - 16:50 (2h 30m)
```
4. Buscar por usuario: "usuario1@edd.com"
5. Ver solo sesiones de ese usuario
6. Analizar patrones de uso
7. Exportar a JSON para análisis externo
8. Generar reporte gráfico

12.5. Ejemplo 5: Carga Masiva de Contactos (Grupal)

1. Grupo prepara archivo JSON:

```
{
  "usuarios": [
    {
      "email": "estudiante1@edd.com",
      "contactos": [
        "estudiante2@edd.com",
        "estudiante3@edd.com"
      ]
    },
    {
      "email": "estudiante2@edd.com",
      "contactos": [
        "estudiante1@edd.com",
        "profesor@edd.com"
      ]
    }
  ]
}
```

2. Root hace login
3. Selecciona "Carga Masiva de Contactos"
4. Elige archivo JSON
5. Sistema procesa:
 - Valida usuarios existen
 - Agrega a listas circulares
 - Actualiza grafo de contactos
6. Confirmación: " 5 contactos agregados"
7. Generar Reporte de Grafo"
8. Ver visualización de red

12.6. Ejemplo 6: Reaccionar a Mensaje en Comunidad

1. Usuario se une a "Proyectos"
2. Entra a ver mensajes
3. Ve publicación:

[ID: 5] usuario@edd.com - "Fase 3 completa"
Reacciones: 3

4. Clic en " Reaccionar"

5. Ingresar ID: 5
6. Sistema incrementa contador
7. Ahora muestra: Reacciones: 4 ”
8. Actualización visible para todos

13. Preguntas Frecuentes - Fase 3

13.1. ¿Qué es el Blockchain y por qué se usa?

Blockchain es una cadena de bloques enlazados mediante hashes criptográficos. Se usa para:

- Garantizar inmutabilidad de correos enviados
- Crear registro auditable de comunicaciones
- Prevenir alteración de datos históricos
- Demostrar integridad mediante proof-of-work

13.2. ¿Cómo funciona el proof-of-work?

1. Sistema crea bloque con datos del correo
2. Busca número (nonce) que al agregarlo al bloque
3. Genere hash que inicie con ”000”
4. Esto requiere múltiples intentos (trabajo)
5. Una vez encontrado, bloque es válido
6. Se agrega a la cadena

13.3. ¿Qué ventaja tiene el Árbol de Merkle sobre Árbol B?

Árbol de Merkle:

- Verificación criptográfica de integridad
- Detecta modificaciones automáticamente
- Hash raíz representa todo el conjunto
- Más seguro para datos críticos

Árbol B:

- Mejor para búsquedas frecuentes
- Más nodos por nivel
- No garantiza integridad

EDDMail usa ambos: B para búsqueda, Merkle para verificación.

13.4. ¿Cuánto comprime realmente LZW?

Depende del contenido:

- Textos repetitivos: 40-60 % reducción
- Correos formales: 25-40 % reducción
- Mensajes cortos: 10-25 % reducción
- Textos aleatorios: poco o nada

13.5. ¿El Control de Logueo afecta rendimiento?

No significativamente:

- Registro es $O(1)$ al inicio y fin de sesión
- No hay operaciones durante uso normal
- Lista simple es eficiente para este caso
- Búsqueda solo cuando root consulta

13.6. ¿Por qué Grafo no dirigido para contactos?

Porque la relación de contacto es bidireccional:

- Si A tiene a B de contacto, B tiene a A
- Arista representa conexión mutua
- Facilita visualización de red social
- Permite análisis de comunidades

13.7. ¿Se puede modificar un bloque en el blockchain?

Técnicamente sí, pero:

- Modificar un bloque cambia su hash
- Bloques posteriores apuntan al hash anterior
- Habría que reinar todos los bloques siguientes
- Computacionalmente prohibitivo
- Por eso blockchain es inmutable en práctica

13.8. ¿Merkle Tree se reconstruye cada vez?

Sí, cada vez que cambian los favoritos:

- Al marcar nuevo favorito
- Al desmarcar favorito
- Garantiza que raíz sea actual
- Costo: $O(n)$ en reconstrucción
- Beneficio: verificación $O(\log n)$

14. Solución de Problemas - Fase 3

14.1. Blockchain muy lento

Causa: Dificultad de minería muy alta

Solución:

- Reducir prefijo requerido (ej: "00.^{en} vez de "000")
- Ajustar en función `ValidarProofOfWork`
- Balance: seguridad vs rendimiento

14.2. Merkle no detecta cambios

Posibles causas:

- Árbol no se reconstruyó tras cambio
- Hash no se calculó correctamente
- Puntero a correo incorrecto

Verificar:

- Llamar `ConstruirArbolMerkleDesdeCorreos`
- Validar función hash SHA-256
- Generar reporte para inspeccionar

14.3. Compresión LZW falla

Síntomas: Error al guardar archivo

Verificar:

- Permisos de escritura en directorio destino
- Ruta de archivo válida
- Espacio en disco suficiente
- Función `ComprimirLZW` retorna resultado

14.4. Control de Logueo no registra salidas

Causa: Usuario no cierra sesión correctamente

Solución:

- Implementar cierre automático al cerrar ventana
- Detectar señal `OnClose`
- Actualizar timestamp de salida
- Validar que registro se completa

14.5. Grafo de contactos no se genera

Verificar:

- Graphviz instalado correctamente
- Comando `dot` accesible
- Estructura de grafo poblada
- Formato DOT válido

Probar:

```
dot -Tpng grafo_contactos.dot -o grafo_contactos.png
```

14.6. Carga masiva de contactos falla

Causas comunes:

- JSON mal formado
- Usuarios no existen en sistema
- Emails con formato incorrecto

Solución:

- Validar JSON con parser online
- Crear usuarios antes de cargar contactos
- Verificar formato de emails
- Revisar logs de error del sistema

15. Tips de Uso - Fase 3

15.1. Optimización

- Blockchain: Evitar minería innecesaria ajustando dificultad
- Merkle: Reconstruir solo cuando necesario
- LZW: Usar para correos largos (mejor compresión)
- Logueo: Exportar periódicamente para limpiar
- Grafo: Carga masiva para grupos grandes

15.2. Seguridad

- Verificar integridad con Merkle antes de acciones críticas
- Auditar logueo para detectar accesos inusuales
- Blockchain protege histórico de modificaciones
- No compartir archivos comprimidos sin validar

15.3. Mantenimiento

- Generar reportes regularmente
- Exportar logueo para análisis externo
- Validar integridad de blockchain periódicamente
- Hacer respaldo de estructura de grafo
- Documentar cambios en estructura

16. Integración Grupal

16.1. Conformación de Grupos

- 3 integrantes por grupo
- Cada integrante implementa en su proyecto
- Funcionalidad: Carga masiva de contactos y grafo

16.2. Responsabilidades

Cada miembro debe:

1. Implementar grafo de contactos
2. Implementar carga masiva desde JSON
3. Agregar contactos manualmente (interfaz)
4. Generar reporte de grafo con Graphviz
5. Documentar implementación (capturas)

16.3. Manual de Integración

El manual grupal debe incluir:

- Capturas de cada integrante mostrando:
 - Interfaz de carga masiva
 - Archivo JSON usado
 - Proceso de carga
 - Reporte de grafo generado
 - Estructura en código
- Explicación de implementación
- Formato JSON acordado
- Casos de prueba realizados

17. Conclusión

EDDMail Fase 3 culmina el proyecto integrando tecnologías avanzadas:

- **Blockchain:** Inmutabilidad y trazabilidad
- **Merkle Tree:** Verificación criptográfica
- **Compresión LZW:** Optimización de almacenamiento
- **Control de Logueo:** Auditoría de accesos
- **Grafos:** Visualización de redes sociales
- **Reacciones:** Interactividad en comunidades

El sistema demuestra aplicación práctica de:

- Estructuras de datos fundamentales (Fases 1 y 2)
- Estructuras avanzadas (Fase 3)

- Algoritmos de seguridad (hashing, proof-of-work)
- Compresión de datos
- Teoría de grafos
- Trabajo colaborativo grupal

Los reportes Graphviz permiten visualizar el funcionamiento interno de todas las estructuras, facilitando comprensión y depuración.

18. Soporte

- GitHub: <https://github.com/JoseArt777/-EDD-1S2025-202100305.git>
- Email: iosealexander40@outlook.com
- Manual Técnico: Documentación de implementación detallada
- Manual Integración: Guía de trabajo grupal con capturas
- Auxiliar Sección C: MarItzep (GitHub)

EDDMail Fase 3
Universidad de San Carlos de Guatemala
Estructuras de Datos - 2025
Proyecto Completo: Fases 1, 2 y 3